

Ontologies Are So Back: Why AI Agents Are Reviving the Semantic Web

AI engineers are rediscovering ontologies as a way to keep probabilistic agents in deterministic boundaries.



RICHARD MACMANUS

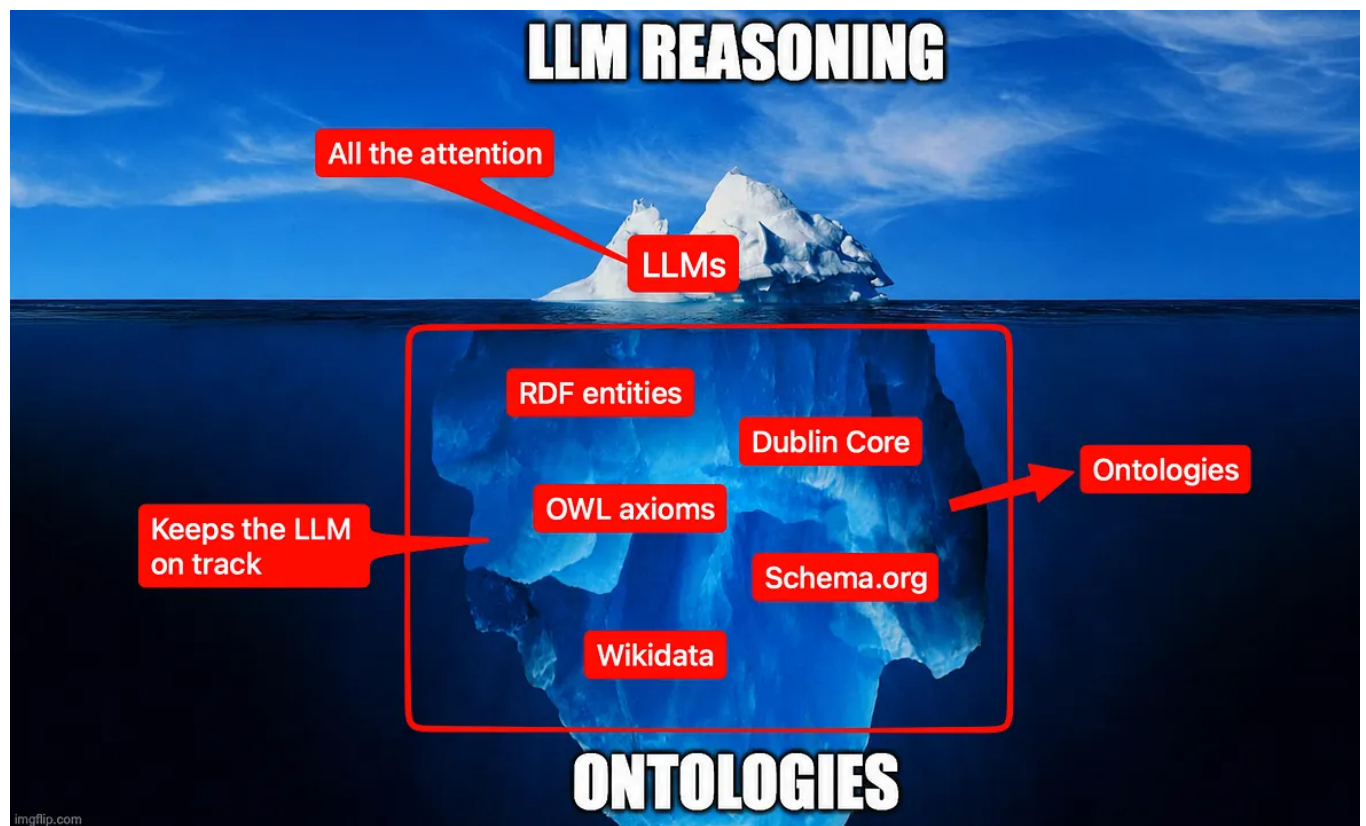
JUL 30, 2026



73

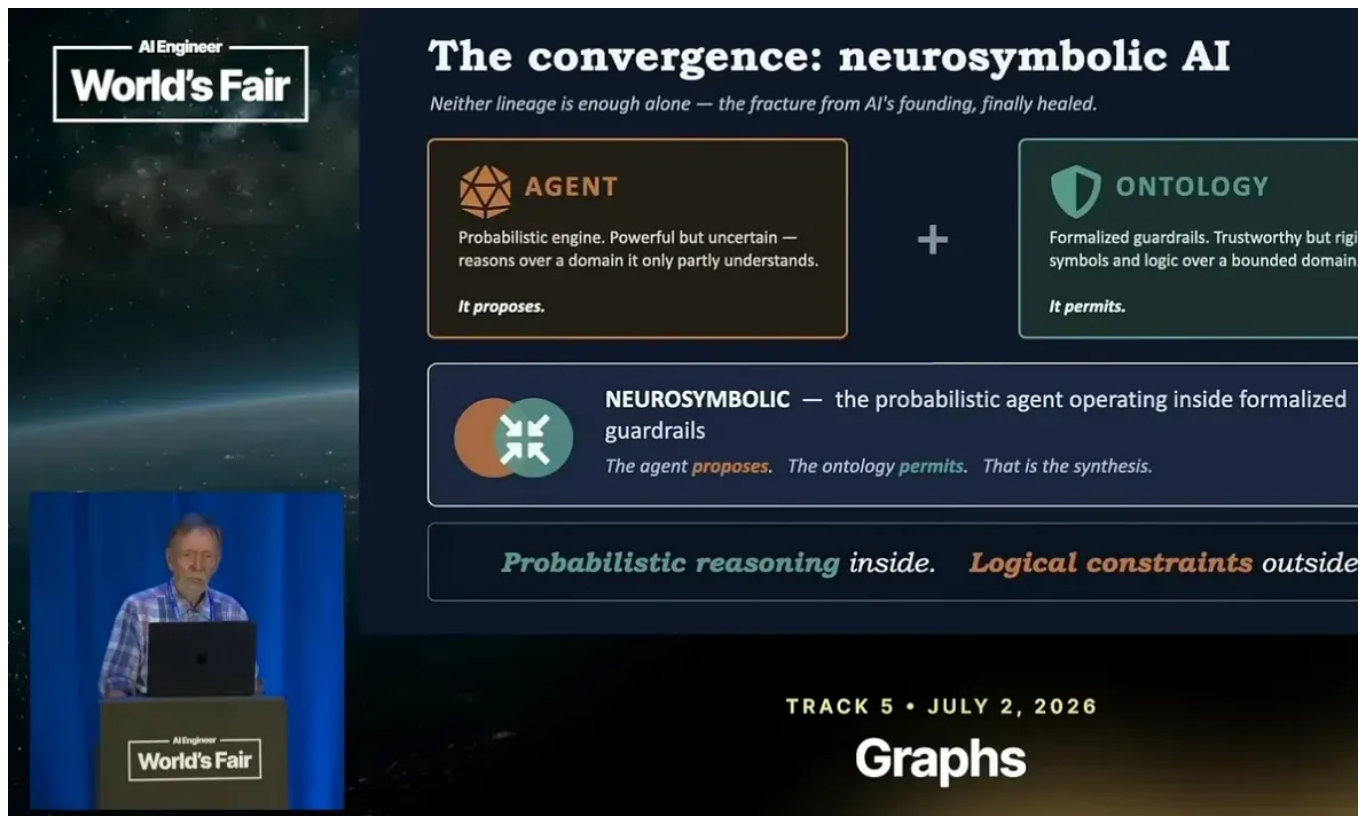


2



One of the most watched videos from the recent AI Engineer World's Fair is a [21 minute talk by Frank Coyle](#), a professor of computer science who currently teaches generative AI and LLMs at UC Berkeley. Drawing on his decades of experience, Coyle **re-introduced the concept and practice of ontologies to today's AI engineers**.

Coyle argued that while LLMs are very effective at providing probabilistic reasoning, for agentic systems to be truly effective they need “logical guardrails” — by which he means ontologies.



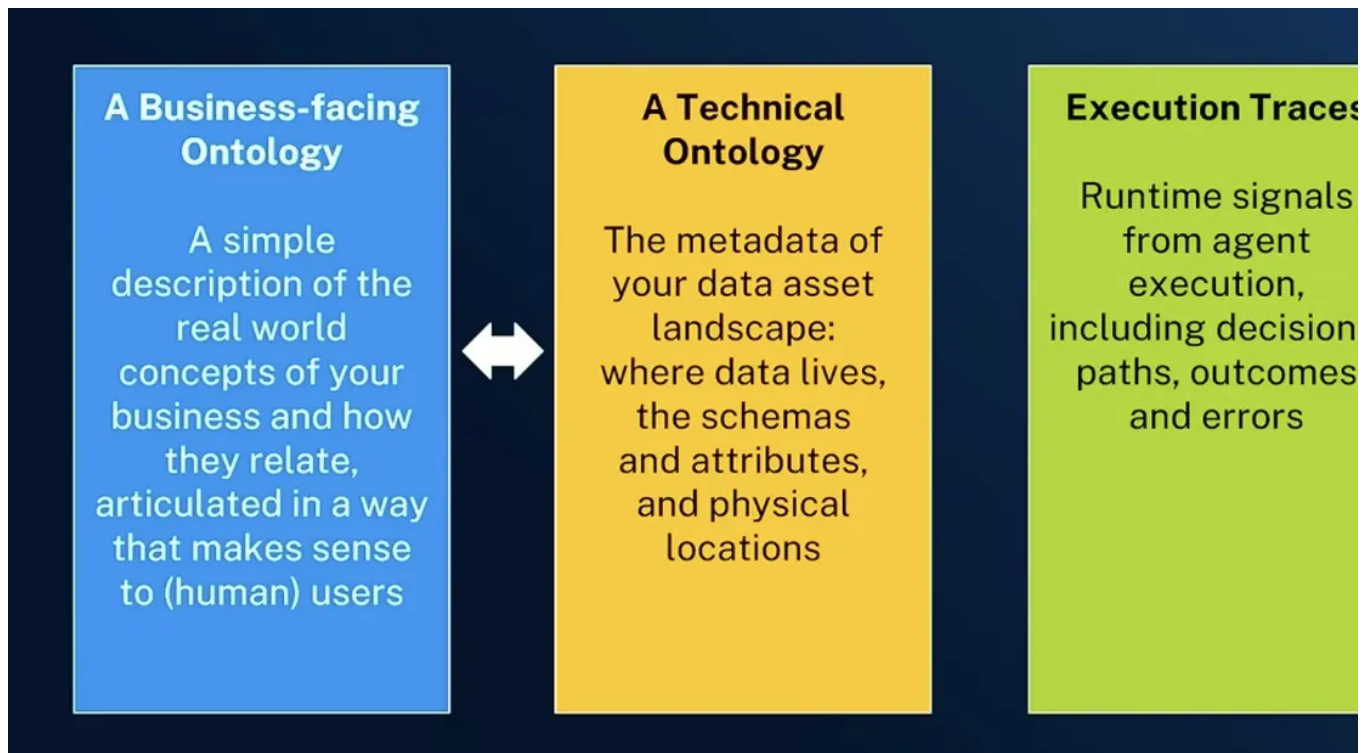
UC Berkeley professor Frank Coyle speaking at AIEWF 2026.

In computer science, an ontology is “a description of data structure – of classes, properties, and relationships in a domain of knowledge” (as nicely [defined by Onto Semantic Technologies](#)). **Coyle himself defined an ontology as simply “data as graphs.”** He added that ontologies as a concept go right back to Aristotle, and have been used throughout the history of Artificial Intelligence.

The company Neo4j, known for its graph database systems, is also using ontologies in its agentic products.

In [a keynote session at AIEWF](#), Neo4j CEO **Emil Eifrem** talked about **three different types of ontologies to enable a “smarter shared substrate” to run agents at scale**. The first is a business-facing ontology, describing the key concepts in an organization; next is a technical ontology, which Eifrem described as “all the metadata of all the data sources and data assets in your enterprise ecosystem” and finally execution traces, which are “the runtime signals out of your agent.”

Ontology-based Semantic Layer



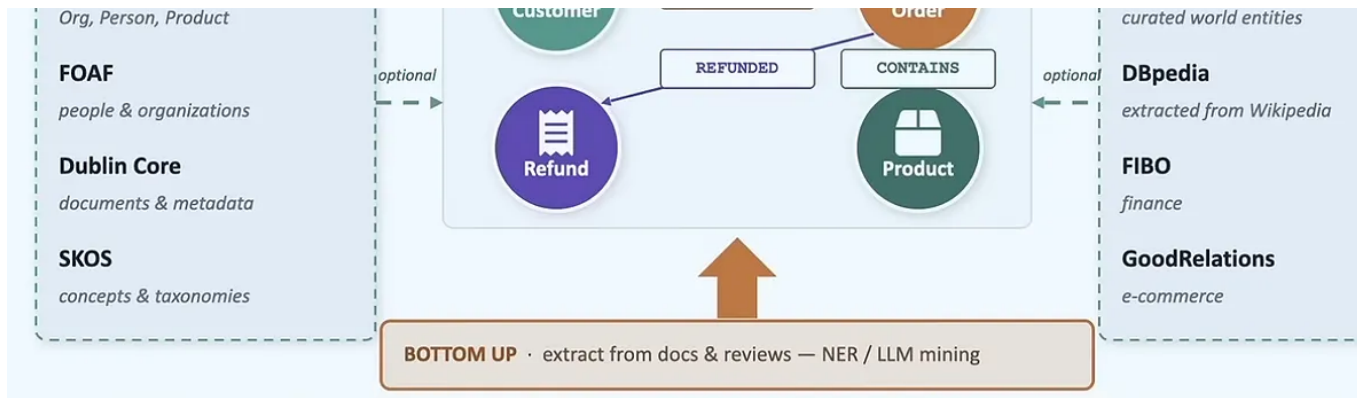
Neo4j's ontology-based semantic layer

What's Old is New Again: the Semantic Web

Frank Coyle thinks **web ontologies** are especially useful in building agentic systems. He mentioned Schema.org, FOAF, Dublin Core and other ontologies familiar to web developers — or at least, those of a certain vintage. He also mentioned “augmentation technologies” like RDFS (Resource Description Framework Schema) and OWL (Web Ontology Language).

One benefit of using these established ontologies is that they're already in the training data of LLMs, so developers can just prompt for them — much better than reinventing ontologies from first principles.





“This stuff has been out there underlying a lot of what we already do,” Coyle said. **“So take advantage of these things that already exist.”**

As an example, he showed a Claude agent loop which used an ontology to help validate the LLM's reasoning after the tool had run.

AGENTIC LOOP ENGINEERING · the loop, and its two gates

A Claude agent loop: call a tool, check stop_reason, repeat

The model drives itself — each turn it asks for a tool or finishes. Two natural gates surround the tool call.

```

context_list = [user_task]      # the running memory

while True:
    resp = client.messages.create(
        model=MODEL, tools=TOOLS,
        messages=context_list)

    if resp.stop_reason == "tool_use":
        call = get_tool_call(resp) # refund(A-91)
        # ↳ model PREDICTS tool + args;
        #   our code names the block 'call'

        # ▶ GATE 1 — validate the INPUT here

        result = run_tool(call)      # tool runs

        # ▶ GATE 2 — validate the OUTPUT here

        context_list.append(
            tool_result(call.id, result))

    else: print(resp.text); break
  
```

TWO GATES, ONE PURE TOOL

`context_list` — the conversation so far; the loop grows it each turn.

GATE 1 — INPUT
before the tool runs.
validate the call's shape.

Pydantic

GATE 2 — OUTPUT
after the tool runs.
validate the result's coherence.

Ontology

Same pattern both times: *Pydantic guards what goes in, the ontology guards what comes out.* We'll fill both gates next.

Coyle calls the convergence of probabilistic agents with ontologies “neurosymbolic AI.”

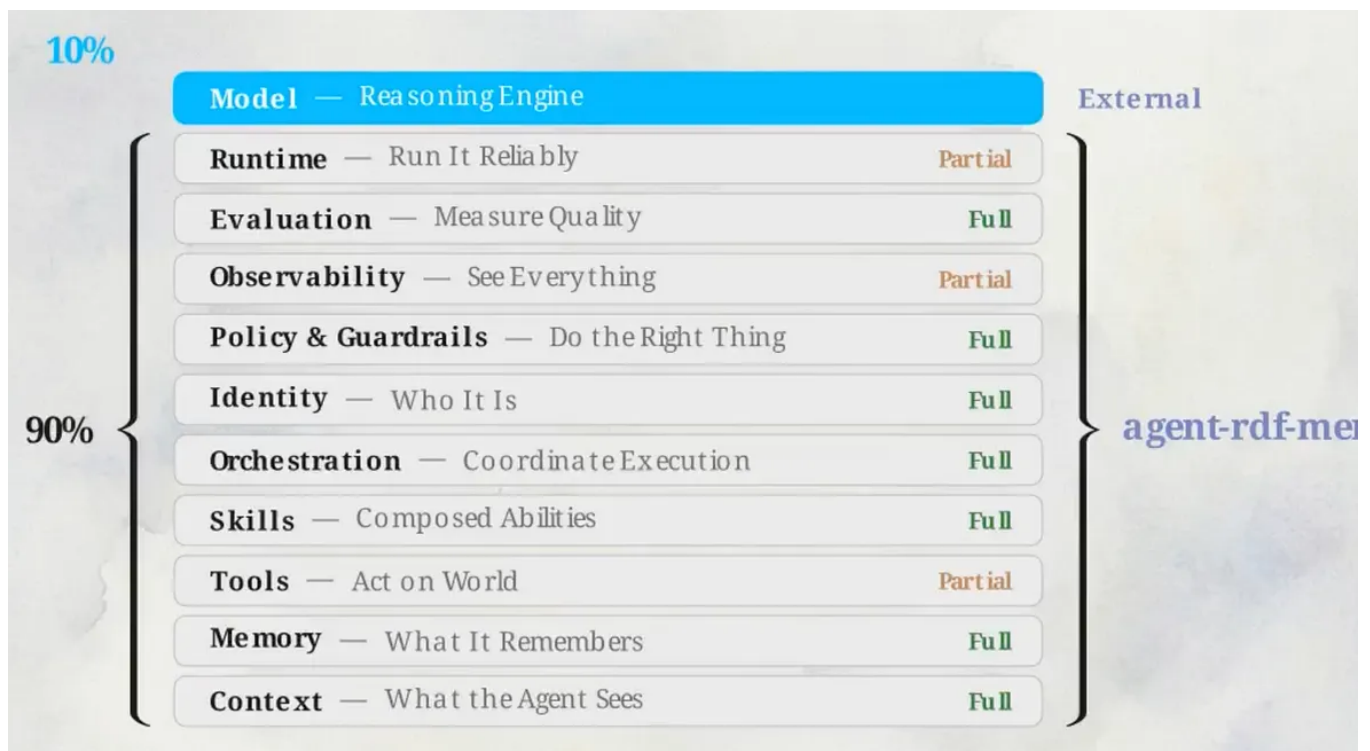
“Sounds pretty fancy,” he said, “but it’s really **neural networks tied into symbolic**”

— rule-based systems come under that category, as do the knowledge graphs we're assembling."

He reiterated that neurosymbolic AI represents "a way to keep the LLM on its guardrails."

The Pros and Cons of Ontologies

Someone who has been steeped in ontologies for many years and is now combining that with AI engineering is **Kingsley Idehen**, who runs a company called [OpenL Software](#). He's been building an "agent engineering stack" that uses Semantic Web technologies — [including](#) an "agent with RDF memory."

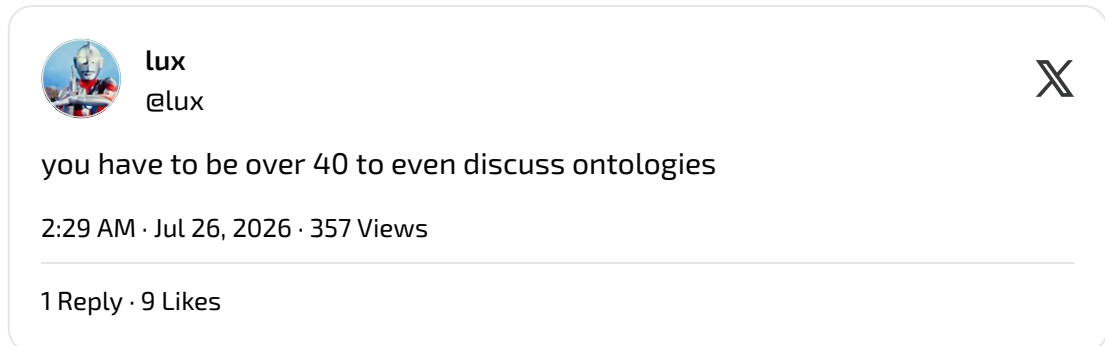


Kingsley Idehen's agent-rdf-memory system.

I asked Idehen to explain to me the benefits of ontologies.

"The beauty of LLMs is that they are powerful processors of language," he replied. **"The beauty of an ontology is that it defines the types of entities and relationships through which language acquires computable context.** Together they bring the expressive power of language to computing's UI/UX stack."

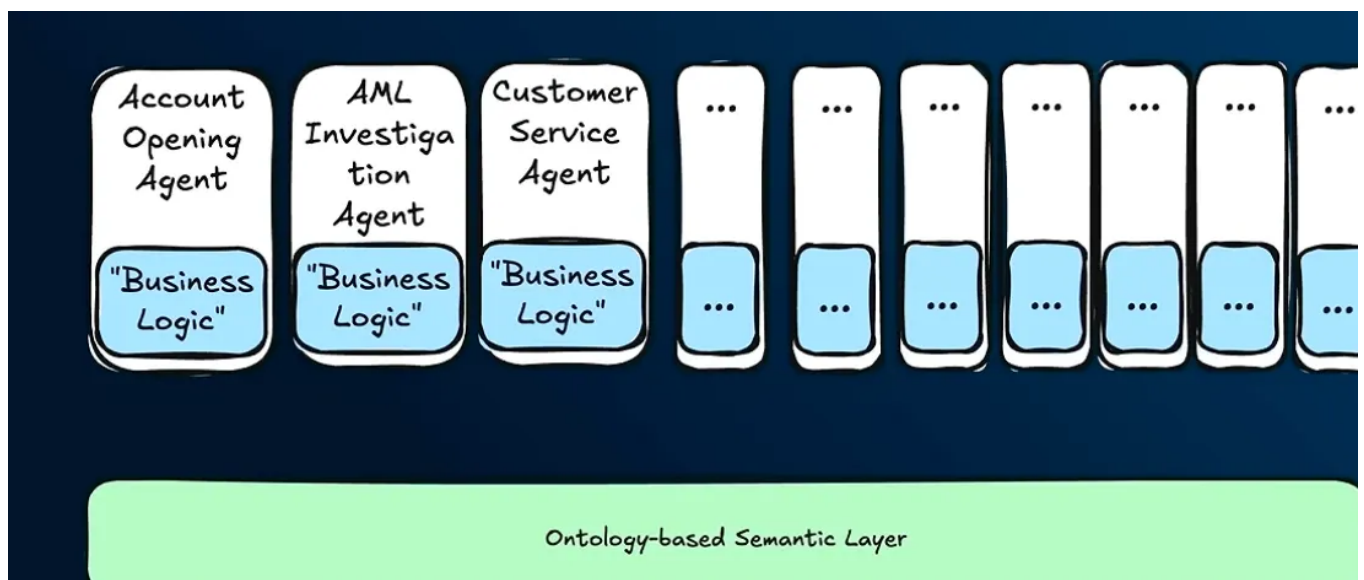
That makes a lot of sense, but if you've been a web developer for a while you'll face the challenges of ontologies: maintenance and keeping them up-to-date. It's with the 1990s and 2000s vision for a "Semantic Web" — which was based on ontologies — never took off.

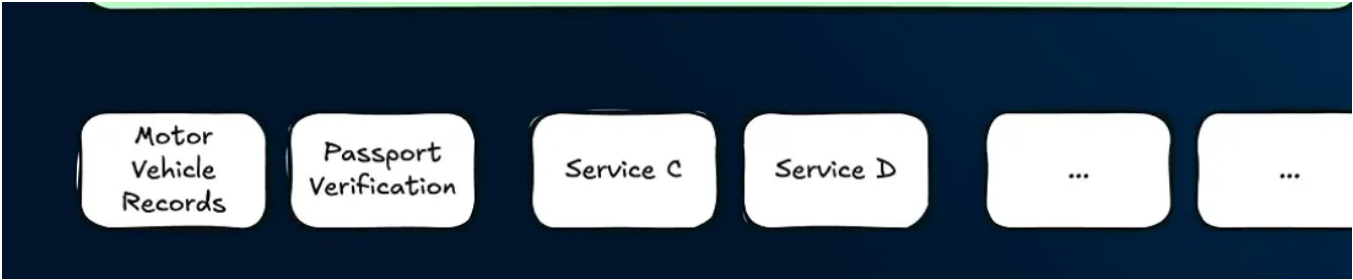


Current AI developer Prasenjit Sarkar offered a potential solution for the maintenance problem on X, [arguing that](#) **“when an agent maintains the ontology part of its own operation**, updating definitions when it encounters edge cases maintenance problem changes character.” It’s still a hard problem though, he a

Despite these issues, the structured nature of ontologies does appear to be a good match with the occasionally wayward tendencies of probabilistic LLMs. **You get the power of LLMs, but ontologies will keep them in check.**

Plus, as Neo4j’s Eifrem explained, ontologies allow us to move from “a world of agents with manually wired data sources” to a new world of “thin agents on a smarter shared ontology-based semantic layer.”





Neo4j's "thin agents" concept, which relies on ontologies.

Loops and Guardrails

Back to Coyle's presentation. He also had a great point about loop engineering, which he noted "has been around forever" in computer science. The problem, of course, is that **loops can break or otherwise "go off the rails."**

Again, this is where an ontology system can act as a guardrail to a probabilistic One of Coyle's slides referred to it as **"a bounded set of rules around an unbounded loop."**

Near the end of his presentation, Coyle demonstrated how to use OWL as a check on the agents. One slide showed that while language can be slippery, "an OWL axiom is a rule a machine enforces."

OWL catches what prose can't pin down		
A sentence in a spec is a hope; an OWL axiom is a rule a machine enforces.		
THE RULE, IN ENGLISH	THE OWL AXIOM	THE ERROR IT CATCHES
An order can be refunded at most once.	<code>:refunded a owl:FunctionalProperty</code>	A second refund on the same order — the double-payout bug.
A buyer and a support-desk employee are never the same account.	<code>:Customer owl:disjointWith :SupportRep</code>	A payout sent to the support desk instead of the buyer.
Order status is one of a fixed set — not free text.	<code>:Status owl:oneOf (paid shipped refunded)</code>	A made-up value like "probably shipped" no tool can read.
A refund must point back to an order that exists.	<code>:refunded rdfs:domain :Order</code>	A refund attached to the wrong object, or a bad call.

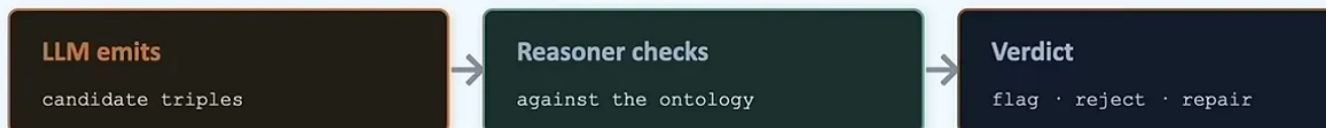
order that exists.

of thing — a baa call.

He also showed how “you can have a reasoner built on ontology, to check [and] the LLM on track — have guardrails to keep it honest.”

The reasoner as a validation layer

The LLM emits candidate triples; a reasoner checks them against the ontology; violations are flagged, rejected, or fed back for repair.



MORE OWL PROPERTY CHARACTERISTICS — each enables a consistency check

owl:TransitiveProperty

$P(a,b) \wedge P(b,c) \rightarrow P(a,c)$

partOf, ancestorOf — validates that chained claims stay consistent.

owl:inverseOf

hasParent $\equiv$ inverse hasChild

an assertion one way implies the other — cross-checks both directions

owl:Symmetric / Asymmetric

siblingOf sym; parentOf asym

asymmetry catches contradictions: $A \text{ parentOf } B \wedge B \text{ parentOf } A$.

owl:propertyDisjointWith

parentOf disjoint siblingOf

the same pair can't hold both relations — flags a category error.

Domain / range typing

rdfs:domain · rdfs:range

the endpoints must be the right kinds — catches malformed links.

Semantic Vibes

Perhaps ontologies are starting to resonate with AI engineers because **a central concern at this time is quality control for loop engineering**. We saw [this debate play out at AIEWF](#), with many conference speakers not willing to go all-in on fully automated “[software factories](#)” just yet. One of the [key learnings from the event](#) was that there need to be guardrails and humans in the loop.

Also it's fascinating to see traditional web technologies make a resurgence in the field of AI engineering, especially after the 2025 trend of “vibe coding” made it seem like anyone could create software. Of course, since then the penny has dropped: we need to maintain that software and make sure it doesn't break! **So in 2026, we're seeing a return to software engineering discipline — including now a revival**

web ontologies as a way to keep probabilistic LLMs honest.

Subscribe to Latent.Space

Thousands of paid subscribers

The AI Engineer newsletter + Top technical AI podcast. How leading labs build Agents, Models, Infr for Science. See <https://latent.space/about> for highlights from Greg Brockman, Andrej Karpathy, C Hotz, Simon Willison, Soumith Chintala et al!

By subscribing, you agree Substack's [Terms of Use](#), and acknowledge its [Information Collection Notice](#) and [Privacy Policy](#).



73 Likes · 2 Restacks

← Previous

Discussion about this post

Comments Restacks



Write a comment...